

COMPUTER SCIENCE TRIPOS Part IB – 2020 – Paper 4

4 Compiler Construction (tgg22)

Consider the following Context Free Grammar

$$S \rightarrow Aa \mid BAb$$

$$A \rightarrow BB \mid c$$

$$B \rightarrow Sd \mid e$$

where $\{a, b, c, d, e\}$ is the set of terminal symbols.

Context Free
Grammar

- (a) Give a right-most derivation of *ecadeb*. [1 mark]

Answer:

$$\begin{aligned} S &\Rightarrow BAb \\ &\Rightarrow BBBb \\ &\Rightarrow BBeb \\ &\Rightarrow BSdeb \\ &\Rightarrow BAadeb \\ &\Rightarrow Bcadeb \\ &\Rightarrow ecadeb \end{aligned}$$

Context Free
Grammar

- (b) Give a left-most derivation of *ecadeb*. [1 mark]

Answer:

$$\begin{aligned} S &\Rightarrow BAb \\ &\Rightarrow eAb \\ &\Rightarrow eBBb \\ &\Rightarrow eSdBb \\ &\Rightarrow eAadBb \\ &\Rightarrow ecadBb \\ &\Rightarrow ecadeb \end{aligned}$$

Parsing

- (c) Compute FIRST and FOLLOW for this grammar. Show your work. [6 marks]

Answer: We noticed in lecture that if there are no ϵ productions (as is the case here), then the calculation of FIRST and FOLLOW can be simplified to the following.

FIRST: We first initialize the sets as follows.

- (i) For each terminal X set $\text{FIRST}(X) = \{X\}$.
- (ii) For each production $X \rightarrow a\alpha$ where a is a terminal, add a to the set $\text{FIRST}(X)$.

We then repeat this step until no more changes can be made:

- For each production $X \rightarrow Y\alpha$ where Y is a nonterminal, add all elements of $\text{FIRST}(Y)$ to the set $\text{FIRST}(X)$.

FOLLOW: We first initialize the sets as follows.

- (i) $\text{FOLLOW}(S) = \{\$ \}$.

— *Solution notes* —

- (ii) For each production $X \rightarrow \alpha Y \beta$ where Y is a nonterminal and $\beta \neq \epsilon$, add all elements of $\text{FIRST}(Y')$ to the set $\text{FOLLOW}(Y)$, where Y' is the first symbol of β .

We then repeat this step until no more changes can be made:

- For each production $X \rightarrow \alpha Y$ where Y is a nonterminal, add all elements of $\text{FOLLOW}(Y)$ to the set $\text{FOLLOW}(X)$.

Using these definitions we can calculate

$$\begin{aligned}\text{FIRST}(S) &= \{c, e\} \\ \text{FIRST}(A) &= \{c, e\} \\ \text{FIRST}(B) &= \{c, e\} \\ \\ \text{FOLLOW}(S) &= \{\$, d\} \\ \text{FOLLOW}(A) &= \{a, b\} \\ \text{FOLLOW}(B) &= \{c, e\}\end{aligned}$$

Parsing

- (d) Is the grammar LL(1)? Justify your answer. [4 marks]

Answer: No. And there is no need to build the parsing tables. It is enough to observe that the grammar is left recursive since $S \Rightarrow BAb \Rightarrow SdAb$, and so we know that construction of LL(1) parsing tables will fail. Note that this is a question about the *grammar*, not the *language* generated by the grammar. (So the question is not asking for a rewrite of the grammar to one that is not left recursive.) Another way to see this is to build the parsing table using FOLLOW and FOLLOW and see that there are multiple entries in some places.

Parsing

- (e) Is the grammar SLR(1)? Justify your answer. [8 marks]

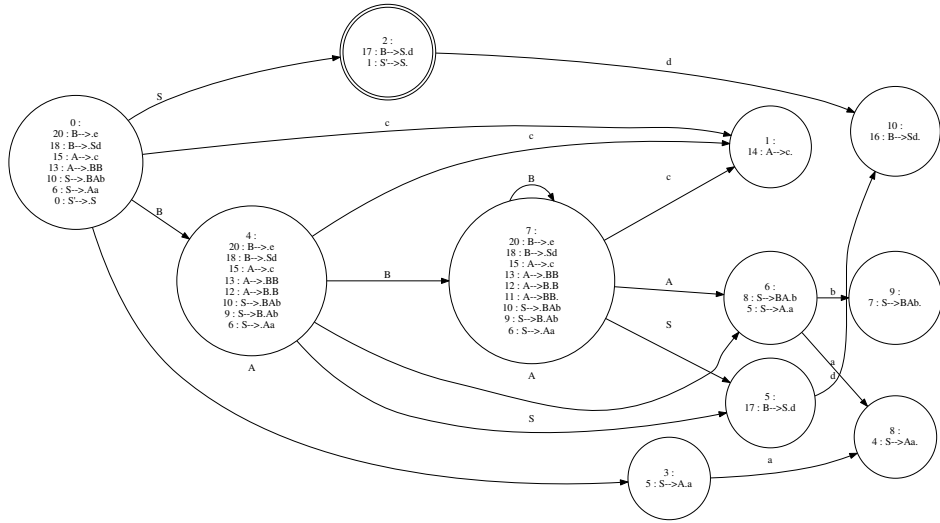
Answer: We must construct the SLR(1) action table and check that there are no shift-reduce or reduce-reduce conflicts. That is,

- (i) Augment the grammar with the production $S' \rightarrow S\$,$
- (ii) construct the NFA,
- (iii) construct the DFA from the NFA
- (iv) check that there are no shift-reduce or reduce-reduce conflicts.

Note that

- (i) A shift-reduce conflict arises only when an NFA state set contains both $[A \rightarrow \alpha \bullet a\beta]$ and $[B \rightarrow \beta \bullet]$ for some nonterminals A, B , and $a \in \text{FOLLOW}(B)$.
- (ii) A reduce-reduce conflict arises only when an NFA state contains both $[A \rightarrow \alpha \bullet]$ and $[B \rightarrow \beta \bullet]$ for some nonterminals A, B and there exists a terminal a such that $a \in \text{FOLLOW}(A)$ and $a \in \text{FOLLOW}(B)$. In particular, this will hold if there is an NFA state that contains both $[A \rightarrow \alpha \bullet]$ and $[A \rightarrow \beta \bullet]$ for some nonterminals A .

The DFA is



By inspection we can see there are no conflicts, so the grammar is SLR(1).
